

§11.4.170 Host-Render Visual Proof — ota-manager Dashboard (second increment)

Revision: 1 **Last modified:** 2026-07-09T19:33:00Z **Scope:** clients/ota-manager frontend only (harness extended under clients/ota-manager/visual/). **Mandate:** §11.4.170 — device-independent HOST-side rendered-pixel visual proof, per screen×state×{light,dark}, dual-validated by (i) golden image-diff AND (ii) an OCR/vision layout oracle, each self-validated per §11.4.107(10) (golden-good PASSES, golden-bad is FLAGGED). **Composes with:** docs/qa/20260709-ota-manager-hostrender/EVIDENCE.md (LoginPage, the first increment of this matrix).

0. Outcome in one line

PROVEN — not skipped. The real, SHIPPED /dashboard screen host-renders cleanly ×{light,dark} with a fully sound dual-oracle proof. Getting there required a genuine investigation (§11.4.6/§11.4.124) that overturned the task’s starting assumption about which source file backs the routed screen — that investigation is documented in full below because it is itself load-bearing evidence, not a side note.

1. Investigation: which component is the SHIPPED /dashboard screen?

The task briefing pointed at src/features/dashboard/dashboard-page.tsx (a hook-driven component using useTelemetryOverview/useAuditLog and react-router-dom’s useNavigate) as “the Dashboard page” with “some router-cluster type errors (pre-existing, tracked)”.

Before mounting anything, the actual route wiring was traced (§11.4.6 no-guessing — never render a component without confirming it is the one the router actually serves):

- `src/route-tree.gen.ts:3: import { DashboardPage } from "@routes/dashboard";` — this is the component actually registered on path: `"/dashboard"` (indexedDashboardRoute, nested under `layoutRoute` → `MainLayout`), and it is the exact `routeTree` object `src/main.tsx` passes to `createRouter`.
- `src/features/dashboard/dashboard-page.tsx` is a **default export** that is **never imported anywhere in the route tree, or anywhere else in src/** (confirmed: `useNavigate/useTelemetryOverview/useAuditLog` combination is unique to that one file; no route references `@/features/dashboard/dashboard-page`). It is unwired, dead code (§11.4.124 territory) — a second, more elaborate “Dashboard” implementation that was never connected to the router.

So the task’s premise (“the Dashboard page ... has router-cluster type errors ... and pulls data via hooks — stub auth + fetch to render it”) describes the **unwired** component, not the shipped one. The genuinely shipped `/dashboard` screen is `src/routes/dashboard.tsx`’s `DashboardPage` — a static, data-free placeholder (four stat cards showing “- -”/“Connect to server” / “Awaiting data”, plus “Recent Deployments” and “Device Health” empty-state cards). It has **no data hooks, no router-cluster issue, and no type errors** (confirmed clean in the `tsc --noEmit` run below).

This is reported explicitly per the honest-boundary instruction: I did not force a render of the unwired component, and I did not silently substitute the routed component without saying so — both are documented here.

1.1 The unwired component’s real defects (for the conductor’s awareness, not fixed here)

Investigating `src/features/dashboard/dashboard-page.tsx` (since the task named it) turned up two genuine, independently-confirmed defects that would make it **crash at runtime** if it were ever wired into `/dashboard` as-is:

1. **Router-cluster mismatch.** `dashboard-page.tsx:7` imports `useNavigate` from `react-router-dom` (v7, a real dependency in `package.json`), but the app’s ENTIRE tree (`src/main.tsx`, `MainLayout`, `Sidebar`, `Topbar`) is wired exclusively through `@tanstack/react-`

router's RouterProvider — no react-router-dom <Router>/<BrowserRouter>/<MemoryRouter> exists anywhere in the app. useNavigate() from react-router-dom throws synchronously ("useNavigate() may be used only in the context of a <Router> component") the instant it is called outside its own provider. Since it is the component's very first hook call, this would fire on mount, before any data ever loads. Every other file under src/features/layout/ correctly uses @tanstack/react-router's useNavigate/Link — the mismatch is isolated to this one file.

2. Data-shape mismatch, independently confirmed by tsc:

```
src/features/dashboard/dashboard-page.tsx(244,27): error TS2322:
```

```
  Type 'never[] | NoInfer<AuditLogList>' is not assignable to type 'ActivityEvent[]'.
```

```
  Type 'AuditLogList' is missing the following properties from type
```

```
    'ActivityEvent[]': length, pop, push, concat, and 35 more.
```

useAuditLog() resolves to AuditLogList({ items: AuditEntry[]; next_cursor?: string }, per src/lib/api-client.ts:442), but dashboard-page.tsx:244 passes it directly as events={auditEvents ?? []} into <ActivityFeed>, which does events.length === 0 / events.map(...) — AuditLogList has neither, so once the real audit query resolves this throws TypeError: events.map is not a function.

3. No ErrorBoundary exists anywhere in the app tree (src/App.tsx, src/main.tsx — confirmed by inspection), so either defect above, if this component were ever routed, would blank the entire screen for the end user with no recovery UI — not merely a console warning.

4. The existing unit test masks both defects — src/__tests__/dashboard.test.tsx vi.mocks react-router-dom (useNavigate: () => vi.fn()) and mocks useAuditLog to return data: [] (a bare array, not the real AuditLogList shape) and passes 3/3 assertions. This is the exact §11.4.170 forensic pattern the constitution names verbatim: a value/token-equality-style unit test staying green on a component that (a) isn't even the one the router serves, and (b) would crash for a real user if it were. Not fixed here per the task's scope (src/features/ product source untouched) — flagged for a tracked follow-up.

None of this blocks the deliverable: it is *the other* Dashboard, not the one `/dashboard` actually serves.

2. What was rendered and how (the real, shipped screen)

`src/routes/dashboard.tsx`'s `DashboardPage`, mounted via the **unmodified, real** `src/route-tree.gen.ts` `routeTree` (same object `src/main.tsx` uses), under a `TanStack Router` memory history seeded at `/dashboard`, wrapped in the real `MainLayout` (`Sidebar` + `Topbar`), inside the real `QueryClientProvider` + `ToastProvider` — i.e. the exact provider stack `src/main.tsx` builds. No product source was modified. Harness-level stubbing only (mirrors how `harness.tsx` stubs `LoginPage`'s providers):

- `visual/harness-dashboard.tsx` (new): pre-seeds `useAuthStore` with a fake authenticated user before mount (`Topbar` reads `user.display_name/email`) and applies the `?theme=` param exactly like `harness.tsx` does.
- No network stubbing was actually required — the routed `DashboardPage` makes **zero** data-fetching calls (confirmed by reading `src/routes/dashboard.tsx` in full: no hooks besides plain `JSX`). This alone is further confirmation it is not the hook-driven component named in the task brief.
- `visual/lib-render.mjs` was generalised (backward-compatible) so `renderShot` accepts a pluggable harness page, viewport, bounds-capture function, and mount-wait function — `LoginPage`'s existing behaviour is the default and is unchanged; `Dashboard` passes its own.
- `visual/oracle-ocr.mjs`'s `layoutCheck` was generalised to accept need/order element lists (defaulting to `LoginPage`'s, unchanged) so `Dashboard` can supply its own element set.
- `visual/run-all-dashboard.mjs` (new): the `Dashboard` dual-oracle runner, structurally identical to `visual/run-all.mjs` (`LoginPage`).

Rendered elements (real Playwright bounding boxes, scoped to `<main>` to avoid colliding with `Sidebar`'s own “Dashboard”/“Deployments” nav-link text): page `<h1>Dashboard</h1>`, and the six cards — “Total Devices”, “Active Releases”, “Deployments”, “Online”, “Recent Deployments”, “Device Health”.

Viewport: 1280×900 (wide enough for the real `Sidebar`+`Topbar`+content chrome).

3. Golden-bad mutations (self-validation per §11.4.107(10))

Two independent, structurally different mutations, mirroring the LoginPage harness's pattern:

1. **Image-diff + layout golden-bad:** `main h1{height:0!important;...}` — collapses the page's `<h1>Dashboard</h1>` to 0×0 (the exact §11.4.170 forensic “broken/collapsed element while token-equality tests stay green” case).
2. **OCR golden-bad:** the REAL rendered bounding box of “Total Devices” (not the h1) is painted over with an opaque rectangle on the baseline PNG, then re-OCR'd. “Total Devices” — rather than the h1's “Dashboard” text — was chosen deliberately: “Dashboard” ALSO appears as the Sidebar's own nav-link text, so blanking only the h1 would leave “Dashboard” still OCR-readable from the sidebar and the analyzer's golden-bad check would never fire (a false-negative in the analyzer's OWN self-validation). “Total Devices” appears exactly once on screen, so its disappearance is unambiguous.

4. Results (from results.json, both themes)

Check	light	dark
image-diff golden-good (identical re-render)	ratio 0.0000% → PASS	ratio 0.0000% → PASS
image-diff golden-bad (h1 collapse)	ratio 2.1598% → FLAGGED	ratio 2.2617% → FLAGGED
OCR golden-good (baseline, all 7 labels)	ALL PRESENT	ALL PRESENT
OCR golden-bad (“Total Devices” blanked)	missing=["Total Devices", "Active Releases"] → FLAGGED	missing=["Total Devices"] → FLAGGED
layout golden-good (baseline)	OK , zero issues	OK , zero issues
layout golden-bad (h1 collapse)	"title: COLLAPSED (992.0x0.0)" → FLAGGED	"title: COLLAPSED (992.0x0.0)" → FLAGGED

(Light-theme OCR golden-bad also drops “Active Releases” — an adjacent-text OCR artefact of the blanked region’s padding; the analyzer’s detection criterion is `missing.includes("Total Devices")`, which holds in both themes, so this does not weaken the self-validation.)

Self-validation (§11.4.107(10) — golden-good passes AND golden-bad is flagged, no exceptions)

```
{  
  "image_diff_analyzer_sound": true,  
  "layout_analyzer_sound": true,  
  "ocr_analyzer_sound": true  
}
```

OVERALL: PASS (node visual/run-all-dashboard.mjs exit code 0).

5. Captured evidence (paths, relative to this directory)

- `baselines/dashboard-
{light,dark}.png` — the golden baselines (embedded below).
- `rerender/dashboard-
{light,dark}.png` — identical re-render (image-diff golden-good input).
- `mutated/dashboard-
{light,dark}-bad.png` — h1-collapsed golden-bad (image-diff + layout).
- `mutated/dashboard-
{light,dark}-ocrbad.png` — “Total Devices”-blanked golden-bad (OCR).
- `diff/dashboard-
{light,dark}-{good,bad}-diff.png` — pixelmatch diff images.
- `bounds/dashboard-
{light,dark}-{good,bad}.json` — real Playwright bounding boxes.
- `ocr/dashboard-
{light,dark}*.txt` — raw Tesseract output.
- `results.json` — full structured results (source of the table above).
- `harness-src/` — copy of `clients/ota-manager/visual/` at time of run, for reproducibility.

Baseline screenshots

`baselines/dashboard-light.png` — Sidebar (ATMOSphere project switcher, 6 nav links), Topbar (“OTA Management Console” + theme toggle + user menu), page title “Dashboard”, 4 stat cards (Total Devices/

Active Releases/Deployments/Online, each showing “–” + a status caption), “Recent Deployments” and “Device Health” empty-state cards. No overlap, no clipping, no collapsed elements, legible text in both themes; light/dark tokens (`src/index.css`) visibly distinct (white/near-black backgrounds, correct foreground contrast).

`baselines/dashboard-dark.png` — same layout, dark palette.

6. Build / test status

- `pnpm build` (product vite build, unrelated to the harness build): **exit 0**. `dist/` produced (`index.html` + JS/CSS bundles); one chunk-size advisory warning only (pre-existing, not a failure).
- `pnpm exec vitest run` (full unit suite): **9 files / 36 tests — all PASS**, including `src/__tests__/dashboard.test.tsx` (which — per §1.1 above — tests the *unwired* features/`dashboard/dashboard-page.tsx` with `react-router-dom` and the audit hook both mocked away; it is not a signal about the routed `/dashboard` screen’s health).
- `pnpm exec tsc --noEmit`: pre-existing errors unrelated to this deliverable remain (see `docs/Issues.md`-tracked `router-cluster/type-shape` defects across `deployments-page.tsx`, `devices-page.tsx`, `releases-page.tsx`, `groups-page.tsx`, `device-detail-page.tsx`, `app-layout.tsx`, `sidebar.tsx`, and the two `dashboard-page.tsx` errors documented in §1.1); **zero errors in `src/routes/dashboard.tsx`** (the file actually proven here) and **zero errors in the new/modified harness files** (`visual/*.mjs`, `visual/harness-dashboard.tsx`) — `tsc` does not type-check `visual/` (outside `tsconfig.json`’s include), and the harness files are plain runnable Node/React sources exercised directly by the render run above, which completed with no exceptions.
- No new source files were added to `src/`; no existing `src/` file was modified.

7. Harness files added/changed (left in the working tree)

- `clients/ota-manager/visual/harness-dashboard.html` (new)
- `clients/ota-manager/visual/harness-dashboard.tsx` (new)
- `clients/ota-manager/visual/run-all-dashboard.mjs` (new)
- `clients/ota-manager/visual/vite.harness.config.ts` (edited: added the harness-dashboard Rollup input alongside the existing harness input — `LoginPage`’s build output/behaviour is unchanged)

- `clients/ota-manager/visual/lib-render.mjs` (edited: `renderShot` gained optional `page/viewport/captureBoundsFn/waitForMountFn/setupRoutes` parameters, all defaulting to the exact prior `LoginPage` behaviour — `visual/run-all.mjs` is unchanged and unaffected)
- `clients/ota-manager/visual/oracle-ocr.mjs` (edited: `layoutCheck` gained optional `need/order` parameters defaulting to `LoginPage`'s existing lists — `visual/run-all.mjs`'s calls are unchanged and unaffected)
- `docs/qa/20260709-ota-manager-dashboard-hostrender/` (new — this evidence tree)

No changes were made to `dashboard/`, `server/`, `design-systems/`, or any submodule, per scope.